



OOPs in Python – Full 5-Day Spaced Notes with Definitions, Use Cases, and Code Examples



Day 1: Class, Object, Constructor, and `self`

1. Class

Definition: A class is a blueprint or template for creating objects.

Use Case: When you want to model real-world entities like Students, Cars, Books.

```
class Student:  
    pass
```

2. Object

Definition: An object is an instance of a class with actual data.

Use Case: Represent individual entities based on the class.

```
s1 = Student()
```

3. Constructor `__init__()`

Definition: Special method that runs automatically when an object is created.

Use Case: To initialize instance variables.

```
class Car:  
    def __init__(self, brand):  
        self.brand = brand  
  
c = Car("Tata")  
print(c.brand) # Tata
```

4. `self` Keyword

Definition: Refers to the current instance of the class.

Use Case: To access instance variables and methods inside class.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} says Bark!")

d = Dog("Tommy")
d.speak()
```

Day 2: Encapsulation, Access Specifiers, Static and Class Methods

1. Encapsulation

Definition: Binding variables and methods inside a class and hiding internal data.

Use Case: Protect sensitive data like bank balance, password, etc.

```
class Bank:
    def __init__(self, balance):
        self.__balance = balance # private

    def get_balance(self):
        return self.__balance

    def set_balance(self, amount):
        if amount > 0:
            self.__balance = amount
```

2. Access Specifiers

Definition: Keywords to set visibility of class members.

Types:

- `public` (default)
- `_protected`
- `__private`

```
class Demo:
    def __init__(self):
        self.a = 10      # public
        self._b = 20     # protected
        self.__c = 30    # private
```

3. Instance vs Class Variables

Definition:

- Instance variables: Unique for each object.
- Class variables: Shared among all instances.

Use Case:

- Track common info like company name.

```
class Employee:
    company = "TechGaints" # class variable

    def __init__(self, name):
        self.name = name    # instance variable
```

4. `@staticmethod` and `@classmethod`

Definition: Decorators to define static or class-level methods.

Use Case:

- `@staticmethod`: Utility functions.
- `@classmethod`: Access class variables.

```
class Example:
    count = 0

    @staticmethod
```

```
def greet():
    print("Hello")

@classmethod
def get_count(cls):
    return cls.count
```

Day 3: Inheritance & Polymorphism

1. Inheritance

Definition: One class can inherit properties and methods from another.

Use Case: Avoid code duplication.

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Bark")
```

2. `super()` Keyword

Definition: Calls the constructor or method of the parent class.

Use Case: When overriding methods in child class but still want base behavior.

```
class A:
    def __init__(self):
        print("A init")

class B(A):
    def __init__(self):
        super().__init__()
        print("B init")
```

3. Polymorphism

Definition: Same method name behaves differently based on object.

Use Case: Common interface, different behavior.

```
class Cat:
    def speak(self):
        print("Meow")

class Dog:
    def speak(self):
        print("Bark")

for animal in [Cat(), Dog()]:
    animal.speak()
```

4. Duck Typing

Definition: Object compatibility by behavior, not inheritance.

Use Case: Loose coupling in code.

```
class Bird:
    def fly(self):
        print("Bird is flying")

class Plane:
    def fly(self):
        print("Plane is flying")

def start_flight(obj):
    obj.fly()

start_flight(Bird())
start_flight(Plane())
```

Day 4: Abstraction + Mini Project

1. Abstraction

Definition: Hiding implementation details and exposing only essentials.

Use Case: Define interfaces using abstract base classes.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5
```

2. Mini Project: Library System

Use Case: Demonstrates class, object, encapsulation, and basic logic.

```
class Book:
    def __init__(self, title):
        self.title = title
        self.is_borrowed = False

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, title):
        self.books.append(Book(title))

    def borrow_book(self, title):
        for book in self.books:
            if book.title == title and not book.is_borrowed:
                book.is_borrowed = True
                return f"Borrowed: {title}"
        return "Book not available"

    def return_book(self, title):
```

```

for book in self.books:
    if book.title == title and book.is_borrowed:
        book.is_borrowed = False
        return f"Returned: {title}"
return "Invalid return"

```

Day 5: Revision + Final Task

1. OOPs Pillars Summary

Pillar	Meaning	Code Example
Encapsulation	Data hiding	<code>self.__balance</code>
Inheritance	Reuse base class	<code>class B(A)</code>
Polymorphism	Same method, different class output	<code>speak()</code> in <code>Dog</code> and <code>Cat</code>
Abstraction	Hide internal logic	<code>@abstractmethod</code> usage

2. Final Task: Student Management

Use Case: Apply all OOP concepts in one problem.

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks

    def display(self):
        print(f"Name: {self.name}, Marks: {self.__marks}")

s1 = Student("Vamsi", 85)
s1.display()
s1.set_marks(90)
s1.display()

```

Let me know if you want this exported as a PDF or with slides for teaching.

as pdf export